Gym Management System

Submitted

Ashutosh Yadav (S20251010002)

Y . Yaswanth kumar (S20240010264)

Rithwik joshi (S20240010157)

Bharath (S20240010167)

Course Name: DBMS

Institution: IIIT SRI CITY

Submission Date :

Problem Statement:

Managing a gym involves keeping track of many things like members, trainers, attendance, payments, and health details. Doing all this manually or using spreadsheets can be confusing, slow, and full of mistakes. It's hard to find information quickly or know if a member's plan has expired. That's why we need a proper system to manage everything in one place.

Objectives :

- The main objective of this project is to make it easy for gym owners to manage records and run their gym smoothly.
- Storing and managing member and trainer details.
- Recording attendance with check-in and check-out times.
- Tracking health data like weight, height, and BMI.
- Managing membership plans and payments.
- Helping admins and trainers get useful reports.
- Making the system simple, fast, and reliable.

Requirement Analysis:

Functional Requirements:

These are the main features the system should provide:

Add, update, and delete member and trainer details

Record attendance with check-in and check-out times

Track health data like weight, height, and BMI

Manage membership plans and payment records

Generate reports for admin and trainers

Renew memberships and calculate remaining days

Non-Functional Requirements:

These are qualities that make the system reliable and easy to use:

Accuracy: Data should be correct and consistent.

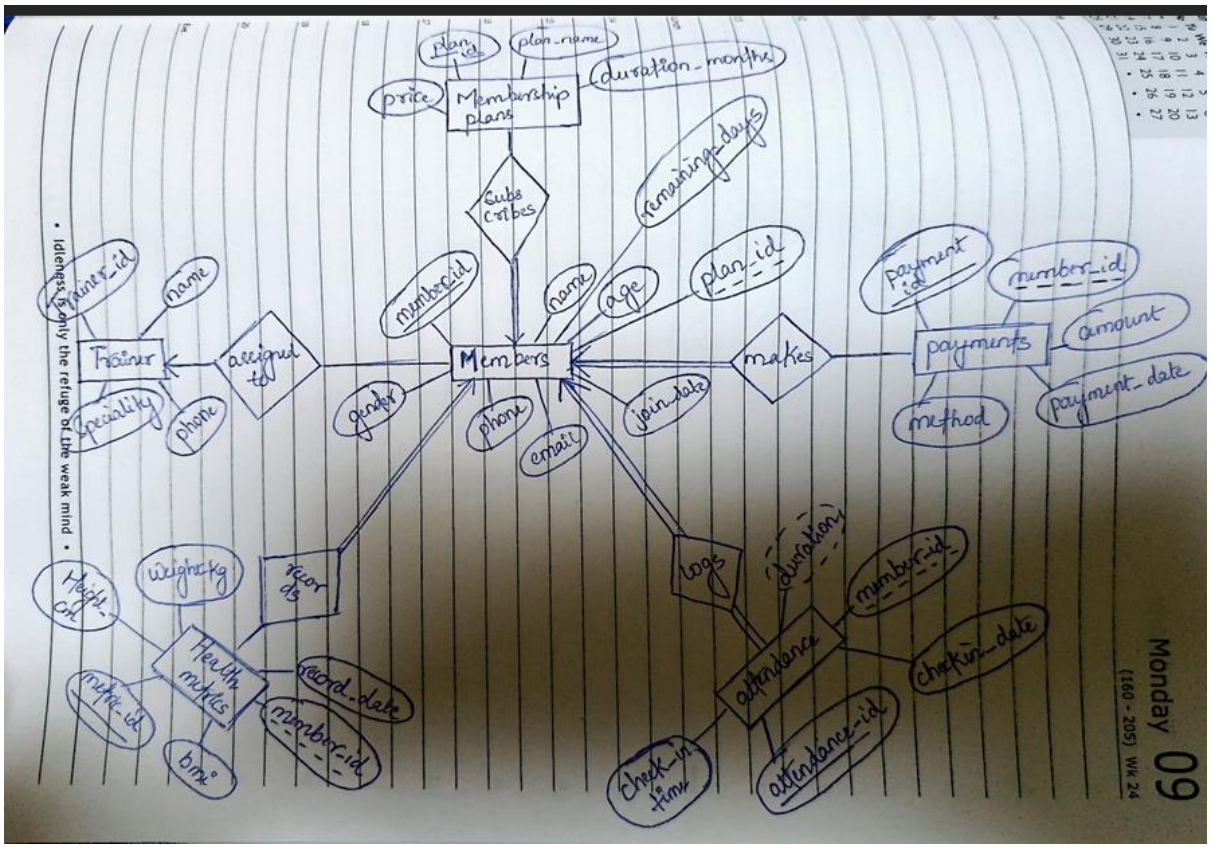
Security: Only authorized users can access or change data

Speed: The system should respond quickly to user actions.

Simplicity: Easy to use for admins, trainers, and staff.

Scalability: Can handle more members and data in the future.

ER Diagram:



Types of users:

In our database management system, we selected three types of users based on how a real gym operates:

Admin

Manages the entire system

Adds and updates member and trainer details

Handles payments and membership plans

Views all reports and data

Trainer

Helps all members during workouts

Can view health data like weight, height, and BMI

Supports members with fitness advice and progress tracking

Member

Views their own attendance and health data

Checks membership status and renewal dates

Can track personal progress and updates

Normalization:

Normalization is the process of organizing data to reduce redundancy and improve data integrity. We applied the following steps to design our Gym DB tables:

Initially, we had one table(reasoning for why we split Members into Members and MembershipPlans):

Members(
member_id,
name,
age,
gender,
phone,
email,
join_date,
plan_id,
plan_name,
duration_months,
price,

remaining_days)

member_id	name	age	gender	phone	email	join_date	plan_id	plan_name	duration_months	price	remaining_days
1	Sweety	22	Female	9876500018	sweety@example.com	2025-01-05	2	Standard	3	2499.00	NULL
2	Rajesh	30	Male	9876500002	rajesh@example.com	2025-02-10	1	Basic	1	999.00	NULL
3	Rohit Patel	29	Male	9876500012	rohitpatel@example.com	2025-03-15	3	Premium	6	4499.00	NULL
4	Priya	28	Female	9876500017	priya@example.com	2025-04-20	2	Standard	3	2499.00	NULL
5	Manideppy	22	Female	9876500003	manideppy@example.com	2025-06-12	3	Premium	6	4499.00	NULL
6	Dinesh	35	Male	9876500010	dinesh@example.com	2025-07-01	2	Standard	3	2499.00	NULL
7	Rupa	34	Female	9876500019	rupa@example.com	2025-07-18	1	Basic	1	999.00	NULL
8	Surya	26	Male	9876500007	surya@example.com	2025-08-03	3	Premium	6	4499.00	NULL
9	Peerthi	26	Female	9876500015	peerthi@example.com	2025-08-20	2	Standard	3	2499.00	NULL
10	Bharath	28	Male	9876500005	bharath@example.com	2025-09-05	1	Basic	1	999.00	NULL
11	Abhi	25	Male	9876500001	abhi@example.com	2025-09-12	3	Premium	6	4499.00	NULL
12	Ramaya	21	Female	9876500014	ramaya@example.com	2025-09-25	2	Standard	3	2499.00	NULL
13	Rohit	31	Male	9876500008	rohit@example.com	2025-10-01	1	Basic	1	999.00	NULL
14	Karthik Reddy	30	Male	9876500020	karthik@example.com	2025-10-20	4	Couple Pack	3	3999.00	NULL
15	Anjali Reddy	27	Female	9876500021	anjali@example.com	2025-10-20	4	Couple Pack	3	3999.00	NULL

Functional dependencies:

1. member_id → name, age, gender, phone, email, join_date, plan_id, remaining_days
2. plan_id → plan_name, duration_months, price
3. phone → member_id
4. email → member_id

These are the candidate key FDs.

Normal forms:

The table is clearly in **1NF** since the attributes are atomic in nature.

There is no partial dependency, because single attribute(member_id) is the primary key.

Therefore the table is in **2NF**.

Now, for 3NF there shouldn't be any transitive dependency, but:

member_id → plan_id

plan_id → plan_name, duration_months, price

here, clearly, we can see there is transitive dependency(plan_id → plan_name, duration_months, price, a non-prime to non-prime FD).

This is violation of 3NF.

To solve this we split the table into:

Members(member_id, name, age, gender, phone, email, join_date, plan_id, remaining_days).

MembershipPlans(plan_id, plan_name, duration_months, price).

```
MariaDB [GymDB]> select * from  Members;
+-----------+--------------+-----+--------+------------+----------------------+------------+---------+----------------+
| member_id | name         | age | gender | phone      | email                | join_date  | plan_id | remaining_days |
+-----------+--------------+-----+--------+------------+----------------------+------------+---------+----------------+
|         1 | Sweety       |  22 | Female | 9876500018 | sweety@example.com   | 2025-01-05 |       2 |             92 |
|         2 | Rajesh       |  30 | Male   | 9876500002 | rajesh@example.com   | 2025-02-10 |       1 |           NULL |
|         3 | Rohit Patel  |  29 | Male   | 9876500012 | rohitpatel@example.com | 2025-03-15 |       3 |           NULL |
|         4 | Priya        |  28 | Female | 9876500017 | priya@example.com    | 2025-04-20 |       2 |           NULL |
|         5 | Manideppy    |  22 | Female | 9876500003 | manideppy@example.com | 2025-06-12 |       3 |           NULL |
|         6 | Dinesh       |  35 | Male   | 9876500010 | dinesh@example.com   | 2025-07-01 |       2 |           NULL |
|         7 | Rupa         |  34 | Female | 9876500019 | rupa@example.com     | 2025-07-18 |       1 |           NULL |
|         8 | Surya        |  26 | Male   | 9876500007 | surya@example.com    | 2025-08-03 |       3 |           NULL |
|         9 | Peerthi      |  26 | Female | 9876500015 | peerthi@example.com  | 2025-08-20 |       2 |           NULL |
|        10 | Bharath      |  28 | Male   | 9876500005 | bharath@example.com  | 2025-09-05 |       1 |           NULL |
|        11 | Abhi         |  25 | Male   | 9876500001 | abhi@example.com     | 2025-09-12 |       3 |           NULL |
|        12 | Ramaya       |  21 | Female | 9876500014 | ramaya@example.com   | 2025-09-25 |       2 |           NULL |
|        13 | Rohit        |  31 | Male   | 9876500008 | rohit@example.com    | 2025-10-01 |       1 |           NULL |
|        14 | Karthik Reddy |  30 | Male   | 9876500020 | karthik@example.com  | 2025-10-20 |       4 |           NULL |
|        15 | Anjali Reddy |  27 | Female | 9876500021 | anjali@example.com   | 2025-10-20 |       4 |           NULL |
+-----------+--------------+-----+--------+------------+----------------------+------------+---------+----------------+
15 rows in set (0.004 sec)
```

```
MariaDB [GymDB]>
MariaDB [GymDB]> -- Membership plans
MariaDB [GymDB]> SELECT * FROM MembershipPlans;
+---------+-------------+-----------------+---------+
| plan_id | plan_name   | duration_months | price   |
+---------+-------------+-----------------+---------+
|       1 | Basic       |               1 |  999.00 |
|       2 | Standard    |               3 | 2499.00 |
|       3 | Premium     |               6 | 4499.00 |
|       4 | Couple Pack |               3 | 3999.00 |
+---------+-------------+-----------------+---------+
4 rows in set (0.000 sec)
```

Now, both tables are also in BCNF, since the FDs member_id and Plan_id are primary keys for thier respective relations.

Database Schema:

The GymDB database contains multiple tables to manage members, trainers, plans, attendance, payments, and health records. Below is the schema based on the actual SQL code used.

MembershipPlans Table

Column Name	Data Type	Description
plan_id	INT	Unique ID for each plan

Column Name	Data Type	Description
plan_name	VARCHAR(50)	Name of the plan (e.g., Couple Pack)
duration_months	INT	Duration of the plan in months
price	DECIMAL(8,2)	Cost of the plan

Members Table

Column Name	Data Type	Description
member_id	INT	Unique ID for each member
name	VARCHAR(100)	Member's full name
age	INT	Member's age
gender	VARCHAR(10)	Gender
phone	VARCHAR(15)	Contact number
email	VARCHAR(100)	Email address
join_date	DATE	Date of joining
plan_id	INT	Linked to MembershipPlans
remaining_days	INT	Days left in membership validity

Trainers Table

Column Name	Data Type	Description
trainer_id	INT	Unique ID for each trainer
name	VARCHAR(100)	Trainer's full name
specialty	VARCHAR(100)	Area of expertise
phone	VARCHAR(15)	Contact number

Payments Table

Column Name	Data Type	Description
payment_id	INT	Unique ID for each payment
member_id	INT	Linked to Members

Column Name	Data Type	Description
amount	DECIMAL(8,2)	Payment amount
payment_date	DATE	Date of payment
method	VARCHAR(50)	Payment method (e.g., Cash, UPI)

Attendance Table

Column Name	Data Type	Description
attendance_id	INT	Unique ID for each attendance record
member_id	INT	Linked to Members
checkin_date	DATE	Date of check-in
in_time	TIME	Time of arrival
out_time	TIME	Time of leaving
total_duration	TIME	Duration of stay

HealthMetrics Table

Column Name	Data Type	Description
metric_id	INT	Unique ID for each health record
member_id	INT	Linked to Members
record_date	DATE	Date of health check
weight_kg	DECIMAL(5,2)	Weight in kilograms

Column Name	Data Type	Description
height_cm	DECIMAL(5,2)	Height in centimeters
bmi	DECIMAL(5,2)	Body Mass Index

SQL QUERIES

Here we will see the SQL commands used to create the GymDB database and all related tables.

```
-- Create database and switch to it
CREATE DATABASE GymDB;
USE GymDB;

-- MembershipPlans table
CREATE TABLE MembershipPlans (
  plan_id INT AUTO_INCREMENT PRIMARY KEY,
  plan_name VARCHAR(50),
  duration_months INT,
  price DECIMAL(8,2)
);


-- Members table
CREATE TABLE Members (
  member_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  age INT,
  gender VARCHAR(10),
  phone VARCHAR(15),
```

```
  email VARCHAR(100),
  join_date DATE,
  plan_id INT,
  remaining_days INT,
  FOREIGN KEY (plan_id) REFERENCES MembershipPlans(plan_id)
);

-- Trainers table
CREATE TABLE Trainers (
  trainer_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  specialty VARCHAR(100),
  phone VARCHAR(15)
);

-- Payments table
+TABLE Payments (
  payment_id INT AUTO_INCREMENT PRIMARY KEY,
  member_id INT,
  amount DECIMAL(8,2),
  payment_date DATE,
  method VARCHAR(50),
  FOREIGN KEY (member_id) REFERENCES Members(member_id)
);

-- Attendance table
CREATE TABLE Attendance (
  attendance_id INT AUTO_INCREMENT PRIMARY KEY,
  member_id INT,
  checkin_date DATE,
  in_time TIME,
```

```
  out_time TIME,
  total_duration TIME,
  FOREIGN KEY (member_id) REFERENCES Members(member_id)
);

-- HealthMetrics table
CREATE TABLE HealthMetrics (
  metric_id INT AUTO_INCREMENT PRIMARY KEY,
  member_id INT,
  record_date DATE,
  weight_kg DECIMAL(5,2),
  height_cm DECIMAL(5,2),
  bmi DECIMAL(5,2),
  FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
```

-- updated codess

```
UPDATE HealthMetrics
SET bmi = ROUND(weight_kg / POWER(height_cm / 100, 2), 2);



---- Updates remaining_days after each payment based on plan duration
DELIMITER //
CREATE TRIGGER UpdateRemainingDays
AFTER INSERT ON Payments
FOR EACH ROW
BEGIN
  UPDATE Members m
  JOIN MembershipPlans mp ON m.plan_id = mp.plan_id
  SET m.remaining_days = DATEDIFF(
    DATE_ADD(NEW.payment_date, INTERVAL mp.duration_months MONTH),
```

```
    CURDATE()
  )
  WHERE m.member_id = NEW.member_id;
END;
//
DELIMITER ;
```

Insertion of data

-- Insert Membership Plans

```
INSERT INTO MembershipPlans (plan_name, duration_months, price) VALUES
('Basic', 1, 999.00),
('Standard', 3, 2499.00),
('Premium', 6, 4499.00),
('Couple Pack', 3, 3999.00);
```

-- Insert Members

```
INSERT INTO Members (name, age, gender, phone, email, join_date, plan_id) VALUES
('Sweety', 22, 'Female', '9876500018', 'sweety@example.com', '2025-01-05', 2),
('Rajesh', 30, 'Male', '9876500002', 'rajesh@example.com', '2025-02-10', 1),
('Rohit Patel', 29, 'Male', '9876500012', 'rohitpatel@example.com', '2025-03-15', 3),
('Priya', 28, 'Female', '9876500017', 'priya@example.com', '2025-04-20', 2),
('Manideppy', 22, 'Female', '9876500003', 'manideppy@example.com', '2025-06-12', 3),
('Dinesh', 35, 'Male', '9876500010', 'dinesh@example.com', '2025-07-01', 2),
('Rupa', 34, 'Female', '9876500019', 'rupa@example.com', '2025-07-18', 1),
('Surya', 26, 'Male', '9876500007', 'surya@example.com', '2025-08-03', 3),
('Peerthi', 26, 'Female', '9876500015', 'peerthi@example.com', '2025-08-20', 2),
('Bharath', 28, 'Male', '9876500005', 'bharath@example.com', '2025-09-05', 1),
```

('Abhi', 25, 'Male', '9876500001', 'abhi@example.com', '2025-09-12', 3),

('Ramaya', 21, 'Female', '9876500014', 'ramaya@example.com', '2025-09-25', 2),

('Rohit', 31, 'Male', '9876500008', 'rohit@example.com', '2025-10-01', 1),

('Karthik Reddy', 30, 'Male', '9876500020', 'karthik@example.com', '2025-10-20', 4),

('Anjali Reddy', 27, 'Female', '9876500021', 'anjali@example.com', '2025-10-20', 4);

-- Insert Payments

INSERT INTO Payments (member_id, amount, payment_date, method) VALUES

(1, 2499.00, '2025-01-05', 'UPI'),

(2, 999.00, '2025-02-10', 'Cash'),

(3, 4499.00, '2025-03-15', 'Card'),

(4, 2499.00, '2025-04-20', 'UPI'),

(5, 4499.00, '2025-06-12', 'Card'),

(6, 2499.00, '2025-07-01', 'UPI'),

(7, 999.00, '2025-07-18', 'Cash'),

(8, 4499.00, '2025-08-03', 'Card'),

(9, 2499.00, '2025-08-20', 'UPI'),

(10, 999.00, '2025-09-05', 'Cash'),

(11, 4499.00, '2025-09-12', 'Card'),

(12, 2499.00, '2025-09-25', 'UPI'),

(13, 999.00, '2025-10-01', 'Cash'),

(14, 3999.00, '2025-10-20', 'Card'),

(15, 3999.00, '2025-10-20', 'Card');

-- Insert Trainers

INSERT INTO Trainers (name, specialty, phone) VALUES

('Mohan Mitra', 'Strength Training', '9876543210'),

('Manidep Maddy', 'Cardio & Endurance', '9123456780'),

('Akhil', 'Yoga & Flexibility', '9988776655'),

('Divya', 'Cardio (Female Clients)', '9012345678');

-- Insert Attendance

```
INSERT INTO Attendance (member_id, checkin_date, in_time, out_time, total_duration) VALUES
(1, '2025-11-01', '07:30:00', '08:45:00', '01:15:00'),
(2, '2025-11-01', '06:50:00', '08:00:00', '01:10:00'),
(3, '2025-11-02', '08:15:00', '09:30:00', '01:15:00'),
(4, '2025-11-02', '07:00:00', '08:10:00', '01:10:00'),
(5, '2025-11-03', '07:20:00', '08:35:00', '01:15:00'),
(6, '2025-11-04', '08:00:00', '09:15:00', '01:15:00'),
(7, '2025-11-04', '07:10:00', '08:30:00', '01:20:00'),
(8, '2025-11-05', '06:55:00', '08:05:00', '01:10:00'),
(9, '2025-11-05', '07:40:00', '09:00:00', '01:20:00'),
(10, '2025-11-06', '07:25:00', '08:40:00', '01:15:00'),
(11, '2025-11-06', '06:50:00', '08:00:00', '01:10:00'),
(12, '2025-11-07', '08:10:00', '09:25:00', '01:15:00'),
(13, '2025-11-07', '07:05:00', '08:15:00', '01:10:00'),
(14, '2025-11-08', '06:40:00', '07:55:00', '01:15:00'),
(15, '2025-11-08', '07:15:00', '08:30:00', '01:15:00');
```

Queries, Procedures, Functions, and Triggers:

Queries

```
-- View members and their remaining days
SELECT member_id, name, remaining_days FROM Members;

-- View health metrics with BMI
SELECT m.name, h.record_date, h.weight_kg, h.height_cm, h.bmi
FROM Members m
JOIN HealthMetrics h ON m.member_id = h.member_id;
```

```
-- List payments made by each member
SELECT m.name, p.payment_date, p.amount
FROM Members m
JOIN Payments p ON m.member_id = p.member_id;

-- Attendance summary
SELECT m.name, a.attendance_date, a.in_time, a.out_time, a.duration_minutes
FROM Members m
JOIN Attendance a ON m.member_id = a.member_id;

-- List trainers and their specialties
SELECT trainer_id, name, specialty FROM Trainers;

-- Show active membership plans
SELECT plan_id, plan_name, duration_months, price FROM MembershipPlans;
```

Procedures

```
-- Adds a new member with selected plan and join date
DELIMITER //
CREATE PROCEDURE AddNewMember(
  IN memberName VARCHAR(100),
  IN plan INT,
  IN joinDate DATE
)
BEGIN
  INSERT INTO Members(name, plan_id, join_date)
  VALUES(memberName, plan, joinDate);
END;
//
DELIMITER ;
```

Functions

```
-- Returns BMI based on weight and height
DELIMITER //
CREATE FUNCTION CalculateBMI(weight DECIMAL(5,2), height_cm INT)
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
  RETURN ROUND(weight / POWER(height_cm / 100, 2), 2);
END;
//
DELIMITER ;
```

Triggers

```
-- Updates remaining_days after each payment based on plan duration
DELIMITER //
CREATE TRIGGER UpdateRemainingDays
AFTER INSERT ON Payments
FOR EACH ROW
BEGIN
  UPDATE Members m
  JOIN MembershipPlans mp ON m.plan_id = mp.plan_id
  SET m.remaining_days = DATEDIFF(
    DATE_ADD(NEW.payment_date, INTERVAL mp.duration_months MONTH),
    CURDATE()
  )
  WHERE m.member_id = NEW.member_id;
END;
//
DELIMITER ;
```

```
-- Calculates and updates BMI after each health record is inserted
DELIMITER //
CREATE TRIGGER CalculateBMI
AFTER INSERT ON HealthMetrics
FOR EACH ROW
BEGIN
  UPDATE HealthMetrics
  SET bmi = ROUND(NEW.weight_kg / POWER(NEW.height_cm / 100, 2), 2)
  WHERE metric_id = NEW.metric_id;
END;
//
DELIMITER ;
```

Creating tables and insertion of data

```
MariaDB [GymDB]> USE GymDB;
Database changed
MariaDB [GymDB]>
MariaDB [GymDB]> -- MembershipPlans table
MariaDB [GymDB]> CREATE TABLE MembershipPlans (
    ->   plan_id INT AUTO_INCREMENT PRIMARY KEY,
    ->   plan_name VARCHAR(50),
    ->   duration_months INT,
    ->   price DECIMAL(8,2)
    -> );
Query OK, 0 rows affected (0.044 sec)

MariaDB [GymDB]>
```

```
Query OK, 0 rows affected (0.031 sec)

MariaDB [GymDB]> -- Trainers table
MariaDB [GymDB]> CREATE TABLE Trainers (
    ->   trainer_id INT AUTO_INCREMENT PRIMARY KEY,
    ->   name VARCHAR(100),
    ->   specialty VARCHAR(100),
    ->   phone VARCHAR(15)
    -> );
Query OK, 0 rows affected (0.058 sec)

MariaDB [GymDB]>
MariaDB [GymDB]> -- Payments table
MariaDB [GymDB]> CREATE TABLE Payments (
    ->   payment_id INT AUTO_INCREMENT PRIMARY KEY,
    ->   member_id INT,
    ->   amount DECIMAL(8,2),
    ->   payment_date DATE,
    ->   method VARCHAR(50),
    ->   FOREIGN KEY (member_id) REFERENCES Members(member_id)
    -> );
Query OK, 0 rows affected (0.061 sec)

MariaDB [GymDB]>
```

```
MariaDB [GymDB]> -- Members table
mbers (
  memberMariaDB [GymDB]> CREATE TABLE Members (
    ->   member_id INT AUTO_INCREMENT PRIMARY KEY,
    ->   name VARCHAR(100),
    ->   age INT,
    ->   gender VARCHAR(10),
    ->   phone VARCHAR(15),
    ->   email VARCHAR(100),
    ->   join_date DATE,
    ->   plan_id INT,
    ->   remaining_days INT,
    ->   FOREIGN KEY (plan_id) REFERENCES MembershipPlans(plan_id)
    -> );
Query OK, 0 rows affected (0.031 sec)

MariaDB [GymDB]>
```

```
MariaDB [GymDB]> -- Attendance table
MariaDB [GymDB]> CREATE TABLE Attendance (
    ->   attendance_id INT AUTO_INCREMENT PRIMARY KEY,
    ->   member_id INT,
    ->   checkin_date DATE,
    ->   in_time TIME,
    ->   out_time TIME,
    ->   total_duration TIME,
    ->   FOREIGN KEY (member_id) REFERENCES Members(member_id)
    -> );
Query OK, 0 rows affected (0.058 sec)

MariaDB [GymDB]>
MariaDB [GymDB]> -- HealthMetrics table

  record_date DMariaDB [GymDB]> CREATE TABLE HealthMetrics (
    -> ATE,
  weight_kg  metric_id INT AUTO_INCREMENT PRIMARY KEY,
    ->   member_id INT,
    ->   record_date DATE,
ECIMAL(5,2),
  F    ->   weight_kg DECIMAL(5,2),
    ->   height_cm DECIMAL(5,2),
    ->   bmi DECIMAL(5,2),
    ->   FOREIGN KEY (member_id) REFERENCES Members(member_id)
    -> );
Query OK, 0 rows affected (0.047 sec)

MariaDB [GymDB]> UPDATE HealthMetrics
    -> SET bmi = ROUND(weight_kg / POWER(height_cm / 100, 2), 2);
Query OK, 0 rows affected (0.028 sec)
Rows matched: 0  Changed: 0  Warnings: 0

MariaDB [GymDB]>
```

Insertion of data

```
MariaDB [GymDB]> INSERT INTO Members (name, age, gender, phone, email, join_date, plan_id) VALUES
    -> ('Sweety', 22, 'Female', '9876500018', 'sweety@example.com', '2025-01-05', 2),
    -> ('Rajesh', 30, 'Male', '9876500002', 'rajesh@example.com', '2025-02-10', 1),
    -> ('Rohit Patel', 29, 'Male', '9876500012', 'rohitpatel@example.com', '2025-03-15', 3),
    -> ('Priya', 28, 'Female', '9876500017', 'priya@example.com', '2025-04-20', 2),
('Manideppy', 22    -> ('Manideppy', 22, 'Female', '9876500003', 'manideppy@example.com', '2025-06-12', 3),
    -> ('Dinesh', 35, 'Male', '9876500010', 'dinesh@example.com', '2025-07-01', 2),
    -> ('Rupa', 34, 'Female', '9876500019', 'rupa@example.com', '2025-07-18', 1),
    -> ('Surya', 26, 'Male', '9876500007', 'surya@example.com', '2025-08-03', 3),
    -> ('Peerthi', 26, 'Female', '9876500015', 'peerthi@example.com', '2025-08-20', 2),
    -> ('Bharath', 28, 'Male', '9876500005', 'bharath@example.com', '2025-09-05', 1),
    -> ('Abhi', 25, 'Male', '9876500001', 'abhi@example.com', '2025-09-12', 3),
    -> ('Ramaya', 21, 'Female', '9876500014', 'ramaya@example.com', '2025-09-25', 2),
    -> ('Rohit', 31, 'Male', '9876500008', 'rohit@example.com', '2025-10-01', 1),
    -> ('Karthik Reddy', 30, 'Male', '9876500020', 'karthik@example.com', '2025-10-20', 4),
    -> ('Anjali Reddy', 27, 'Female', '9876500021', 'anjali@example.com', '2025-10-20', 4);
Query OK, 15 rows affected (0.026 sec)
Records: 15  Duplicates: 0  Warnings: 0

MariaDB [GymDB]>
MariaDB [GymDB]> INSERT INTO Payments (member_id, amount, payment_date, method) VALUES
),
(5, 4499.00,     -> (1, 2499.00, '2025-01-05', 'UPI'),
    -> (2, 999.00, '2025-02-10', 'Cash'),
    -> (3, 4499.00, '2025-03-15', 'Card'),
    -> (4, 2499.00, '2025-04-20', 'UPI'),
    -> (5, 4499.00, '2025-06-12', 'Card'),
    -> (6, 2499.00, '2025-07-01', 'UPI'),
    -> (7, 999.00, '2025-07-18', 'Cash'),
    -> (8, 4499.00, '2025-08-03', 'Card'),
    -> (9, 2499.00, '2025-08-20', 'UPI'),
    -> (10, 999.00, '2025-09-05', 'Cash'),
    -> (11, 4499.00, '2025-09-12', 'Card'),
    -> , 2499.00, '2025(12, 2499.00, '2025-09-25', 'UPI'),
    -> (13, 999.00, '2025-10-01', 'Cash'),
    -> (14, 3999.00, '2025-10-20', 'Card'),
    -> (15, 3999.00, '2025-10-20', 'Card');
Query OK, 15 rows affected (0.023 sec)
Records: 15  Duplicates: 0  Warnings: 0

MariaDB [GymDB]> -- Insert Trainers
MariaDB [GymDB]> INSERT INTO Trainers (name, specialty, phone) VALUES
    -> ('Mohan Mitra', 'Strength Training', '9876543210'),
    -> ('Manidep Maddy', 'Cardio & Endurance', '9123456780'),
    -> ('Akhil', 'Yoga & Flexibility', '9988776655'),
    -> ('Divya', 'Cardio (Female Clients)', '9012345678');
Query OK, 4 rows affected (0.025 sec)
Records: 4  Duplicates: 0  Warnings: 0

MariaDB [GymDB]>
```

```
MariaDB [GymDB]>
MariaDB [GymDB]> INSERT INTO Attendance (member_id, checkin_date, in_time, out_time, total_duration) VALUES
    -> (1, '2025-11-01', '07:30:00', '08:45:00', '01:15:00'),
    -> (2, '2025-11-01', '06:50:00', '08:00:00', '01:10:00'),
    -> (3, '2025-11-02', '08:15:00', '09:30:00', '01:15:00'),
    -> (4, '2025-11-02', '07:00:00', '08:10:00', '01:10:00'),
    -> (5, '2025-11-03', '07:20:00', '08:35:00', '01:15:00'),
    -> (6, '2025-11-04', '08:00:00', '09:15:00', '01:15:00'),
    -> (7, '2025-11-04', '07:10:00', '08:30:00', '01:20:00'),
    -> (8, '2025-11-05', '06:55:00', '08:05:00', '01:10:00'),
    -> (9, '2025-11-05', '07:40:00', '09:00:00', '01:20:00'),
    -> (10, '2025-11-06', '07:25:00', '08:40:00', '01:15:00'),
    -> (11, '2025-11-06', '06:50:00', '08:00:00', '01:10:00'),
    -> (12, '2025-11-07', '08:10:00', '09:25:00', '01:15:00'),
    -> (13, '2025-11-07', '07:05:00', '08:15:00', '01:10:00'),
    -> (14, '2025-11-08', '06:40:00', '07:55:00', '01:15:00'),
    -> (15, '2025-11-08', '07:15:00', '08:30:00', '01:15:00');
Query OK, 15 rows affected (0.032 sec)
Records: 15  Duplicates: 0  Warnings: 0

MariaDB [GymDB]>
MariaDB [GymDB]>
```

Trigger

```
MariaDB [GymDB]> DROP TRIGGER IF EXISTS CalculateBMI;
Query OK, 0 rows affected (0.033 sec)

MariaDB [GymDB]> DROP TRIGGER IF EXISTS CalculateBMI;
Query OK, 0 rows affected, 1 warning (0.001 sec)

MariaDB [GymDB]> DELIMITER //
MariaDB [GymDB]> CREATE TRIGGER CalculateBMI
    -> BEFORE INSERT ON HealthMetrics
    -> FOR EACH ROW
    -> BEGIN
    ->   SET NEW.bmi = ROUND(NEW.weight_kg / POWER(NEW.height_cm / 100, 2), 2);
    -> END;
    -> //
MITER ;
Query OK, 0 rows affected (0.055 sec)

MariaDB [GymDB]> DELIMITER ;
MariaDB [GymDB]> INSERT INTO HealthMetrics(member_id, record_date, weight_kg, height_cm)
    -> VALUES (1, CURDATE(), 70, 175);
Query OK, 1 row affected (0.033 sec)

MariaDB [GymDB]> SELECT * FROM HealthMetrics WHERE member_id = 1;
+-----------+-----------+-------------+-----------+-----------+-------+
| metric_id | member_id | record_date | weight_kg | height_cm | bmi   |
+-----------+-----------+-------------+-----------+-----------+-------+
|         2 |         1 | 2025-11-07  |     70.00 |    175.00 | 22.86 |
+-----------+-----------+-------------+-----------+-----------+-------+
1 row in set (0.002 sec)

MariaDB [GymDB]>
```

Trigger for remaining days

```
MariaDB [GymDB]>
MariaDB [GymDB]> DELIMITER //
MariaDB [GymDB]> CREATE TRIGGER UpdateRemainingDays
    -> AFTER INSERT ON Payments
    -> FOR EACH ROW
    -> BEGIN
    ->   UPDATE Members m
    ->   JOIN MembershipPlans mp ON m.plan_id = mp.plan_id
    ->   SET m.remaining_days = DATEDIFF(
    ->     DATE_ADD(NEW.payment_date, INTERVAL mp.duration_months MONTH),
    ->     CURDATE()
    ->   )
    ->   WHERE m.member_id = NEW.member_id;
    -> END;
    -> //
Query OK, 0 rows affected (0.057 sec)
```

Test for remaning days

```
MariaDB [GymDB]>
MariaDB [GymDB]> SELECT member_id, name, remaining_days FROM Members WHERE member_id = 1;
+-----------+--------+----------------+
| member_id | name   | remaining_days |
+-----------+--------+----------------+
|         1 | Sweety |             92 |
+-----------+--------+----------------+
1 row in set (0.001 sec)
```

Query Outputs

```
MariaDB [GymDB]> -- Health metrics with BMI


-- Trainer lisMariaDB [GymDB]> SELECT m.name, h.record_date, h.weight_kg, h.height_cm, h.bmi
    -> FROM Members m
    -> JOIN HealthMetrics h ON m.member_id = h.member_id;
Trainers;

-- Membership plans
SELECT * FROM MembershipPlans;
+--------+-------------+-----------+-----------+-------+
| name   | record_date | weight_kg | height_cm | bmi   |
+--------+-------------+-----------+-----------+-------+
| Sweety | 2025-11-07  |     70.00 |    175.00 | 22.86 |
+--------+-------------+-----------+-----------+-------+
1 row in set (0.026 sec)

MariaDB [GymDB]>
MariaDB [GymDB]> -- Payment history
MariaDB [GymDB]> SELECT m.name, p.payment_date, p.amount
    -> FROM Members m
    -> JOIN Payments p ON m.member_id = p.member_id;
+---------------+--------------+---------+
| name          | payment_date | amount  |
+---------------+--------------+---------+
| Sweety        | 2025-01-05   | 2499.00 |
| Rajesh        | 2025-02-10   |  999.00 |
| Rohit Patel   | 2025-03-15   | 4499.00 |
| Priya         | 2025-04-20   | 2499.00 |
| Manideppy     | 2025-06-12   | 4499.00 |
| Dinesh        | 2025-07-01   | 2499.00 |
| Rupa          | 2025-07-18   |  999.00 |
| Surya         | 2025-08-03   | 4499.00 |
| Peerthi       | 2025-08-20   | 2499.00 |
| Bharath       | 2025-09-05   |  999.00 |
| Abhi          | 2025-09-12   | 4499.00 |
| Ramaya        | 2025-09-25   | 2499.00 |
| Rohit         | 2025-10-01   |  999.00 |
| Karthik Reddy | 2025-10-20   | 3999.00 |
| Anjali Reddy  | 2025-10-20   | 3999.00 |
| Sweety        | 2025-11-07   | 2499.00 |
+---------------+--------------+---------+
16 rows in set (0.001 sec)
```

```
MariaDB [GymDB]> -- Attendance summary
MariaDB [GymDB]> SELECT m.name, a.checkin_date, a.in_time, a.out_time, a.total_duration
    -> FROM Members m
    -> JOIN Attendance a ON m.member_id = a.member_id;
+---------------+--------------+----------+----------+----------------+
| name          | checkin_date | in_time  | out_time | total_duration |
+---------------+--------------+----------+----------+----------------+
| Sweety        | 2025-11-01   | 07:30:00 | 08:45:00 | 01:15:00       |
| Rajesh        | 2025-11-01   | 06:50:00 | 08:00:00 | 01:10:00       |
| Rohit Patel   | 2025-11-02   | 08:15:00 | 09:30:00 | 01:15:00       |
| Priya         | 2025-11-02   | 07:00:00 | 08:10:00 | 01:10:00       |
| Manideppy     | 2025-11-03   | 07:20:00 | 08:35:00 | 01:15:00       |
| Dinesh        | 2025-11-04   | 08:00:00 | 09:15:00 | 01:15:00       |
| Rupa          | 2025-11-04   | 07:10:00 | 08:30:00 | 01:20:00       |
| Surya         | 2025-11-05   | 06:55:00 | 08:05:00 | 01:10:00       |
| Peerthi       | 2025-11-05   | 07:40:00 | 09:00:00 | 01:20:00       |
| Bharath       | 2025-11-06   | 07:25:00 | 08:40:00 | 01:15:00       |
| Abhi          | 2025-11-06   | 06:50:00 | 08:00:00 | 01:10:00       |
| Ramaya        | 2025-11-07   | 08:10:00 | 09:25:00 | 01:15:00       |
| Rohit         | 2025-11-07   | 07:05:00 | 08:15:00 | 01:10:00       |
| Karthik Reddy | 2025-11-08   | 06:40:00 | 07:55:00 | 01:15:00       |
| Anjali Reddy  | 2025-11-08   | 07:15:00 | 08:30:00 | 01:15:00       |
+---------------+--------------+----------+----------+----------------+
15 rows in set (0.001 sec)

MariaDB [GymDB]>
MariaDB [GymDB]> -- Trainer list
MariaDB [GymDB]> SELECT * FROM Trainers;
+------------+---------------+------------------------+------------+
| trainer_id | name          | specialty              | phone      |
+------------+---------------+------------------------+------------+
|          1 | Mohan Mitra   | Strength Training      | 9876543210 |
|          2 | Manidep Maddy | Cardio & Endurance     | 9123456780 |
|          3 | Akhil         | Yoga & Flexibility     | 9988776655 |
|          4 | Divya         | Cardio (Female Clients) | 9012345678 |
+------------+---------------+------------------------+------------+
4 rows in set (0.000 sec)

MariaDB [GymDB]>
MariaDB [GymDB]> -- Membership plans
MariaDB [GymDB]> SELECT * FROM MembershipPlans;
+---------+-------------+-----------------+---------+
| plan_id | plan_name   | duration_months | price   |
+---------+-------------+-----------------+---------+
|       1 | Basic       |               1 |  999.00 |
|       2 | Standard    |               3 | 2499.00 |
|       3 | Premium     |               6 | 4499.00 |
|       4 | Couple Pack |               3 | 3999.00 |
+---------+-------------+-----------------+---------+
4 rows in set (0.000 sec)
```

Test for procedure

```
MariaDB [GymDB]> CALL AddNewMember('Yashwanth', 2, '2025-11-06');
embers;
Query OK, 1 row affected (0.028 sec)

MariaDB [GymDB]> SELECT * FROM Members;
+-----------+--------------+------+--------+------------+-----------------------+------------+---------+----------------+
| member_id | name         | age  | gender | phone      | email                 | join_date  | plan_id | remaining_days |
+-----------+--------------+------+--------+------------+-----------------------+------------+---------+----------------+
|         1 | Sweety       |   22 | Female | 9876500018 | sweety@example.com    | 2025-01-05 |       2 |             92 |
|         2 | Rajesh       |   30 | Male   | 9876500002 | rajesh@example.com    | 2025-02-10 |       1 |           NULL |
|         3 | Rohit Patel  |   29 | Male   | 9876500012 | rohitpatel@example.com | 2025-03-15 |       3 |           NULL |
|         4 | Priya        |   28 | Female | 9876500017 | priya@example.com     | 2025-04-20 |       2 |           NULL |
|         5 | Manideppy    |   22 | Female | 9876500003 | manideppy@example.com | 2025-06-12 |       3 |           NULL |
|         6 | Dinesh       |   35 | Male   | 9876500010 | dinesh@example.com    | 2025-07-01 |       2 |           NULL |
|         7 | Rupa         |   34 | Female | 9876500019 | rupa@example.com      | 2025-07-18 |       1 |           NULL |
|         8 | Surya        |   26 | Male   | 9876500007 | surya@example.com     | 2025-08-03 |       3 |           NULL |
|         9 | Peerthi      |   26 | Female | 9876500015 | peerthi@example.com   | 2025-08-20 |       2 |           NULL |
|        10 | Bharath      |   28 | Male   | 9876500005 | bharath@example.com   | 2025-09-05 |       1 |           NULL |
|        11 | Abhi         |   25 | Male   | 9876500001 | abhi@example.com      | 2025-09-12 |       3 |           NULL |
|        12 | Ramaya       |   21 | Female | 9876500014 | ramaya@example.com    | 2025-09-25 |       2 |           NULL |
|        13 | Rohit        |   31 | Male   | 9876500008 | rohit@example.com     | 2025-10-01 |       1 |           NULL |
|        14 | Karthik Reddy |   30 | Male   | 9876500020 | karthik@example.com   | 2025-10-20 |       4 |           NULL |
|        15 | Anjali Reddy |   27 | Female | 9876500021 | anjali@example.com    | 2025-10-20 |       4 |           NULL |
|        16 | Yashwanth    | NULL | NULL   | NULL       | NULL                  | 2025-11-06 |       2 |           NULL |
+-----------+--------------+------+--------+------------+-----------------------+------------+---------+----------------+
16 rows in set (0.001 sec)

MariaDB [GymDB]>
```

Conclusion:

The Gym Management System database was successfully designed and implemented using core DBMS concepts such as normalization, ER modeling, relational schema design, SQL queries, triggers, and stored procedures. By organizing the data into well-structured tables-Members, Trainers, MembershipPlans, Payments, Attendance, and HealthMetrics. The system ensures accuracy, reduces redundancy, and maintains data integrity.

Through normalization up to BCNF, the database eliminates anomalies and supports efficient data retrieval. The use of triggers automates important tasks such as calculating BMI and updating remaining membership days, while stored procedures simplify repetitive operations like adding new members. Overall, this system provides a reliable solution that helps gym administrators manage daily operations smoothly, improves data consistency, and enhances decision-making through meaningful reports.

Future scope:

We can add a weak entity “FamilyMembers”, in the sense that, if a member is already subscribed to a membership plan and his/her family member wants to join the gym, they shall get certain percentage discount based on the type of membership plan taken by that member. We can use functions and procedure to make this process seamless to integrate in out database.

Contribution :

ER Diagram , Normalization Steps : Ashutosh Yadav (S20251010002)

Database Schema,Triggers,sample data Y . Yaswanth kumar (S20240010264)

Queries, Procedures	Rithwik joshi(S20240010157)
Functions,output Screenshots	Bharath(S20240010167)